```
[root@vbg ~]# seinfo -c
```

A file on your system, a network socket or a process—all of these are instances of Object Classes. A list of file related Object Classes in the default-loaded policy in my system is shown below:

```
blk_file  - Block File
chr_file  - Character File
lnk_file  - Symbolic Links
fifo_file - Named Pipes
file      - Normal Files
sock_file - UNIX domain sockets
filesystem - Partitions etc
dir       - Directories
fd        - File Descriptors
```

Permissions

Permissions on Object Classes constitute the access control restrictions defined in a SELinux security policy. As you can guess, permissions relate to read/write and other fine grained controls. To list various permissions that can be applied to each of the Object Classes in the default loaded policy, issue the following command:

```
[root@vbg ~]# seinfo -c -x
```

As an example, the permissions that can be applied to an instance of the "file" object in the Targeted Policy loaded on my system, are:

```
file
    append
    create
    execute
    write
    relabelfrom
    link
    unlink
    ioctl
    getattr
    setattr
    read
    rename
    lock
    relabelto
    mounton
    quotaon
    swapon
    entrypoint
    execmod
    execute_no_trans
```

The above example means that processes (subjects) can be given permissions to append, create, execute, write, etc. Depending on the permissions specified to a particular subject, a successful access or denial will occur.

For example, you can specify that the ***httpd*** process can create a file in the ***/tmp*** folder, append to a file in the /var/log/ folder and only read from the /var/www/html folder.

If a newly installed Web application tries to create a file in the ***/var/www/html*** folder (assuming it is owned by the 'apache' user and has all the required DAC permissions using ***chmod***/***chown***) it will still get a denial.

The above example clarifies how MAC restrictions prevent unauthorised tampering of data and secure your critical servers.

Attributes

Attributes group together types with similar properties. They make it easier to specify rules in a policy. For example, most applications (http, ftp, squid, mail) create log files. If the system administrator had to create individual rules for ***logrotate***, appending and creating log files, it would involve a lot of repetitive work.

Once a type attribute is specified to a new object, permissions associated with that attribute are applied to the new object, saving a lot of repetitive work. To view a list of all attributes in the current loaded policy, issue the following command:

```
[root@vbg ~]# seinfo -a -x
```

You can see that log files are grouped together under the attribute ***@ttr1335***:

```
@ttr1335
    amavis_var_log_t
    ccs_var_log_t
    ricci_var_log_t
    nscd_log_t
    var_log_ksyms_t
    ntpd_log_t
    sendmail_log_t
    var_log_t
```

We will deal with categories, sensitivities, ***fs_use*** and ***genfscon*** in detail in subsequent articles in this series. In this article we have looked at more building blocks of SELinux security policies. The various basic building blocks of a SELinux Security Policy can be summarised as:

- Users
- Roles
- Types
- Booleans
- Classes (Object Classes)
- Permissions (on Object Classes)
- Attributes
- Port Contexts
- Node Contexts

Access Control rules can be applied as per permissions assigned to classes. These rules can be ummarised as:

- Allow rules—to allow access
- NeverAllow rules—to prohibit access

There are other rules as well—Type Transitions, AuditAllow, DontAudit, etc. In the next article we will explore how these rules are applied for access permissions to objects based on their Security Contexts.

Still to come

- Understanding the Targeted Policy - Part III
- Policy Modules
- MLS and MCS END

By: Varad Gupta

Varad is an open source enthusiast who strongly believes in the open source collaborative model not only for technology but also for business. India's first RHCSS (Red Hat Certified Security Specialist), he has been involved in spreading open source through Keen & Able Computers Pvt Ltd, an open source systems integration company, and FOSTERing Linux, a FOSS training, education and research training centre. The author can be contacted at varad.gupta@fosteringlinux.com